Question 3

```
import random
import math

# ============================================================================
# PART (a): diagnose_hc
# ============================================================================

def first_choice_hc_full(landscape, start):
    """First-Choice HC returning full path (for diagnosis)."""
    current = start
    path = [current]
    n = len(landscape)
    while True:
        improved = False
        neighbours = []
        if current > 0:
            neighbours.append(current - 1)
        if current < n - 1:
            neighbours.append(current + 1)
        for nb in neighbours:
            if landscape[nb] > landscape[current]:
                current = nb
                path.append(current)
                improved = True
                break
        if not improved:
            break
    return current, path


def diagnose_hc(landscape, start):
    """
    Runs First-Choice HC and diagnoses which failure mode was encountered.

    Failure modes:
      1. Local Maximum  — no neighbour is better OR equal.
      2. Plateau        — no strictly better neighbour, but at least one equal.
      3. Ridge          — same state visited twice (oscillation without progress).
    """
    current = start
    visited = [current]
    n = len(landscape)
```

```
    while True:
        neighbours = []
        if current > 0:
            neighbours.append(current - 1)
        if current < n - 1:
            neighbours.append(current + 1)

        # Check for ridge: would moving to any neighbour revisit a state?
        for nb in neighbours:
            if nb in visited:
                failure = "Ridge"
                print(f"  Terminated at state {current+1} with f={landscape[current]}. "
                      f"Failure mode: {failure}")
                print(f"  Path (1-based): {[v+1 for v in visited]}")
                return current, visited, failure

        # Categorise neighbours
        better  = [nb for nb in neighbours if landscape[nb] > landscape[current]]
        equal   = [nb for nb in neighbours if landscape[nb] == landscape[current]]

        if better:
            # Move to first better neighbour (first-choice)
            current = better[0]
            visited.append(current)
        elif equal:
            # Plateau: equal neighbour exists but no improvement
            failure = "Plateau"
            print(f"  Terminated at state {current+1} with f={landscape[current]}. "
                  f"Failure mode: {failure}")
            print(f"  Path (1-based): {[v+1 for v in visited]}")
            return current, visited, failure
        else:
            # Local Maximum: no better, no equal
            failure = "Local Maximum"
            print(f"  Terminated at state {current+1} with f={landscape[current]}. "
                  f"Failure mode: {failure}")
            print(f"  Path (1-based): {[v+1 for v in visited]}")
            return current, visited, failure


# ---------------------------------------------------------------------------
# Three hand-crafted landscapes (each ≥ 5 states, not from Q1/Q2)
# ---------------------------------------------------------------------------
```

```
# Landscape A — LOCAL MAXIMUM
# State:  1   2   3   4   5   6
# f(s):   1   3   2   8   5   4
# HC starts at state 4 (f=8). Both neighbours state 3 (f=2) and state 5 (f=5)
# are strictly lower → pure local maximum. No equal neighbour either.
LANDSCAPE_A = [1, 3, 2, 8, 5, 4]
START_A = 3   # 0-based index of state 4

# Landscape B — PLATEAU
# State:  1   2   3   4   5   6   7
# f(s):   2   4   9   9   9   5   1
# HC starts at state 2. Climbs: 2→3 (f=9). State 3 has neighbour 4 (f=9, equal)
# and neighbour 2 (f=4, lower). No strictly better neighbour → plateau.
LANDSCAPE_B = [2, 4, 9, 9, 9, 5, 1]
START_B = 2   # 0-based index of state 3 (f=9); neighbour state 4 is equal → plateau

# Landscape C — RIDGE
# State:  1   2   3   4   5   6
# f(s):   3   7   5   7   3   1
# HC starts at state 3 (f=5). Neighbours: state 2 (f=7, better) → move there.
# Now at state 2 (f=7). Neighbours: state 1 (f=3, worse), state 3 (f=5, worse).
# State 3 was already visited → ridge detected.
LANDSCAPE_C = [3, 7, 5, 7, 3, 1]
START_C = 2   # 0-based index of state 3


# ============================================================================
# PART (b): N-Queens (N=8) with Stochastic HC + Random Restarts
# ============================================================================

N = 8

def random_board(n=N):
    """Random permutation — one queen per column, random rows."""
    board = list(range(n))
    random.shuffle(board)
    return board


def count_conflicts(board):
    """
    Returns the number of attacking queen pairs.
    Two queens attack if they share the same row or are on the same diagonal.
```

```
    (Column conflicts are impossible by construction — one queen per column.)
    """
    n = len(board)
    conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:                # same row
                conflicts += 1
            if abs(board[i] - board[j]) == abs(i - j): # same diagonal
                conflicts += 1
    return conflicts


def stochastic_hc_nqueens(board, max_steps=1000):
    """
    Stochastic Hill Climbing for N-Queens.
    At each step: pick two random columns; swap their rows only if the swap
    strictly reduces the number of conflicts.
    Returns (final_board, conflicts, steps_taken).
    """
    current = board[:]
    current_conflicts = count_conflicts(current)
    n = len(current)

    for step in range(max_steps):
        if current_conflicts == 0:
            break
        # Pick two distinct columns at random
        i, j = random.sample(range(n), 2)
        # Try the swap
        candidate = current[:]
        candidate[i], candidate[j] = candidate[j], candidate[i]
        new_conflicts = count_conflicts(candidate)
        if new_conflicts < current_conflicts:
            current = candidate
            current_conflicts = new_conflicts

    return current, current_conflicts, step + 1


def solve_nqueens_rrhc(num_restarts, n=N, verbose=True):
    """
    Applies Random Restart HC to N-Queens until a solution is found
    or restarts are exhausted.
```

```
    Returns (solution_board or None, restarts_used, conflicts).
    """
    for restart in range(1, num_restarts + 1):
        board = random_board(n)
        final, conflicts, steps = stochastic_hc_nqueens(board)
        if conflicts == 0:
            if verbose:
                print(f"  Solution found on restart {restart} after {steps} steps.")
            return final, restart, 0
    if verbose:
        print(f"  No solution found in {num_restarts} restarts.")
    return None, num_restarts, conflicts


def print_board(board):
    """Print the board with Q and . characters."""
    n = len(board)
    print("  +" + "-" * (2 * n - 1) + "+")
    for row in range(n):
        line = []
        for col in range(n):
            line.append("Q" if board[col] == row else ".")
        print("  |" + " ".join(line) + "|")
    print("  +" + "-" * (2 * n - 1) + "+")
    print(f"  Board representation: {board}")
    print(f"  Conflicts: {count_conflicts(board)}")


# ============================================================================
# PART (c): Benchmark
# ============================================================================

def benchmark_nqueens(k_values, trials=30, n=N):
    """
    For each k in k_values, run solve_nqueens_rrhc(k) for 'trials' independent
    trials. Record success rate and average restarts when successful.
    """
    results = {}
    for k in k_values:
        successes = 0
        restart_counts = []
        for _ in range(trials):
            board, restarts_used, conflicts = solve_nqueens_rrhc(
                k, n=n, verbose=False)
```

```
            if conflicts == 0:
                successes += 1
                restart_counts.append(restarts_used)
        success_rate = successes / trials
        avg_restarts = (sum(restart_counts) / len(restart_counts)
                        if restart_counts else float('nan'))
        results[k] = (success_rate, avg_restarts)
    return results


# ================================================================================
# MAIN
# ================================================================================

def main():
    random.seed(42)

    # ============================================================================
    print("=" * 65)
    print("  Q3(a) — DIAGNOSING HILL CLIMBING FAILURES")
    print("=" * 65)

    # ── Landscape A: Local Maximum ──────────────────────────────────────
    print("\n--- Landscape A (Local Maximum) ---")
    print(f"  {'State':>6}  {'f(s)':>6}")
    for i, v in enumerate(LANDSCAPE_A):
        print(f"  {i+1:>6}  {v:>6}")
    print(f"  Start: state {START_A+1}")
    diagnose_hc(LANDSCAPE_A, start=START_A)

    # ── Landscape B: Plateau ────────────────────────────────────────────
    print("\n--- Landscape B (Plateau) ---")
    print(f"  {'State':>6}  {'f(s)':>6}")
    for i, v in enumerate(LANDSCAPE_B):
        print(f"  {i+1:>6}  {v:>6}")
    print(f"  Start: state {START_B+1}")
    diagnose_hc(LANDSCAPE_B, start=START_B)

    # ── Landscape C: Ridge ──────────────────────────────────────────────
    print("\n--- Landscape C (Ridge) ---")
    print(f"  {'State':>6}  {'f(s)':>6}")
    for i, v in enumerate(LANDSCAPE_C):
        print(f"  {i+1:>6}  {v:>6}")
    print(f"  Start: state {START_C+1}")
```

```
    diagnose_hc(LANDSCAPE_C, start=START_C)

    # ======================================================================
    print("\n" + "=" * 65)
    print("  Q3(b) — N-QUEENS (N=8) WITH STOCHASTIC HC + RANDOM RESTARTS")
    print("=" * 65)

    random.seed(0)
    solution, restarts_used, conflicts = solve_nqueens_rrhc(100, verbose=True)

    if solution:
        print(f"\n  Restarts needed : {restarts_used}")
        print(f"  Conflicts       : {conflicts}")
        print(f"\n  Visual board:")
        print_board(solution)
    else:
        print("  No solution found in 100 restarts.")

    # ======================================================================
    print("\n" + "=" * 65)
    print("  Q3(c) — BENCHMARK: solve_nqueens_rrhc(k), 30 trials each")
    print("=" * 65)

    k_values = [5, 10, 25, 50, 100]
    random.seed(1)
    results = benchmark_nqueens(k_values, trials=30)

    print(f"\n  {'k':>5}  {'Success Rate':>14}  {'Avg Restarts (when successful)':>32}")
    print("  " + "-" * 55)
    for k in k_values:
        sr, ar = results[k]
        ar_str = f"{ar:.2f}" if not math.isnan(ar) else "N/A"
        print(f"  {k:>5}  {sr:>13.2%}  {ar_str:>32}")


if __name__ == "__main__":
    main()
```

```
================================================================
 Q3(a) DIAGNOSING HILL CLIMBING FAILURES
================================================================

--- Landscape A (Local Maximum) ---
  State    f(s)
      1       1
      2       3
      3       2
      4       8
      5       5
      6       4
 Start: state 4
 Terminated at state 4 with f=8. Failure mode: Local Maximum
 Path (1-based): [4]
```

Local Maximum (Landscape A)

- **Observation:** Start at state 4 ($f = 8$), no neighboring states have $f \geq 8$.
- **Reason HC fails:** First-Choice Hill Climbing halts because there is no improving neighbor; the algorithm cannot move downhill. Without accepting worse moves (like in Simulated Annealing) or performing a random restart, it remains permanently stuck.

```
--- Landscape B (Plateau) ---
  State    f(s)
      1       2
      2       4
      3       9
      4       9
      5       9
      6       5
      7       1
 Start: state 3
 Terminated at state 3 with f=9. Failure mode: Plateau
 Path (1-based): [3]
```

Plateau (Landscape B)

- **Observation:** Start at state 3 ($f = 9$), all neighbors have $f = 9$.
- **Reason HC fails:** First-Choice HC only moves to strictly better states. On a plateau, all neighbors are equal, so it stalls. Even sideways moves wouldn't guarantee progress off the flat region.

```
--- Landscape C (Ridge) ---
   State    f(s)
       1       3
       2       7
       3       5
       4       7
       5       3
       6       1
  Start: state 3
  Terminated at state 2 with f=7. Failure mode: Ridge
  Path (1-based): [3, 2]
```

Ridge (Landscape C)

- **Observation:** Start at state 3 (f = 5), moves to state 2 (f = 7), but may oscillate in a small set of high-value states.
- **Reason HC fails:** Ridge creates cycles; the algorithm cannot "remember" previous steps, so it oscillates without making net progress toward the global maximum.

```
======================================================================
 Q3(b) N-QUEENS (N=8) WITH STOCHASTIC HC + RANDOM RESTARTS
======================================================================
 Solution found on restart 1 after 9 steps.

 Restarts needed : 1
 Conflicts       : 0

 Visual board:
 +---------------+
 |. . . Q . . . .|
 |. Q . . . . . .|
 |. . . . . . . Q|
 |. . . . . Q . .|
 |Q . . . . . . .|
 |. . Q . . . . .|
 |. . . . Q . . .|
 |. . . . . . Q .|
 +---------------+
 Board representation: [4, 1, 5, 0, 6, 3, 7, 2]
 Conflicts: 0
```

```
============================================================
 Q3(c) ? BENCHMARK: solve_nqueens_rrhc(k), 30 trials each
============================================================

   k    Success Rate    Avg Restarts (when successful)
 -------------------------------------------------------
   5         90.00%                              2.33
  10        100.00%                              2.47
  25        100.00%                              3.07
  50        100.00%                              2.13
 100        100.00%                              3.23
```

Analysis:

1. **Reason HC struggles beyond "local optima":**
    - **Large plateaus:** Many configurations with the same number of conflicts create flat regions where stochastic moves may not improve.
    - **Ridges / narrow paths:** Moving toward the solution may require temporary increases in conflicts; basic HC rejects these moves.
2. **Empirical confirmation:** Even with small k (like 5), success is not guaranteed; with larger k (10–100), random restarts reliably find solutions, confirming the effectiveness of restarts to escape flat regions and ridges.
3. **Limitation for very large N (e.g., N = 1000):** Random Restart HC cannot scale efficiently, because the search space grows factorially, and restarts may be insufficient.
4. **More powerful algorithm: Genetic Algorithms** or **Simulated Annealing** can explore the landscape more effectively, accepting temporary increases in conflicts to escape local optima.